



Laboratorio - Create and maintain database objects

☒ review	☐
☐ unidad	mod1

Una vez que el entorno esté listo y la base de datos cargada, procedemos a ejecutar el **.sql**. A continuación, analizaremos las consultas de sql y el resultado de cada una de ellas.

Creación de tablas principales, tablas

```
/* =====  
==  
2. TABLAS PRINCIPALES  
=====  
== */  
  
-- Crear tabla Supplier  
CREATE TABLE Supplier (  
    SupplierID INT PRIMARY KEY IDENTITY(1,1),  
    SupplierName NVARCHAR(100) NOT NULL UNIQUE,  
    Country NVARCHAR(50) NOT NULL,  
    Email NVARCHAR(100),  
    Phone NVARCHAR(20),  
    CreatedDate DATETIME2 DEFAULT GETUTCDATE()  
);
```

```

-- Create tabla Category
CREATE TABLE Category (
    CategoryID INT PRIMARY KEY IDENTITY(1,1),
    CategoryName NVARCHAR(100) NOT NULL UNIQUE,
    Description NVARCHAR(500)
);

-- Create tabla Product con constraints
CREATE TABLE Product (
    ProductID INT PRIMARY KEY IDENTITY(1,1),
    ProductName NVARCHAR(100) NOT NULL,
    CategoryID INT NOT NULL,
    SupplierID INT NOT NULL,
    BasePrice DECIMAL(10,2) NOT NULL,
    StockQuantity INT NOT NULL DEFAULT 0,
    CreatedDate DATETIME2 DEFAULT GETUTCDATE(),
    CHECK (BasePrice > 0),
    CHECK (StockQuantity >= 0),
    FOREIGN KEY (CategoryID) REFERENCES Category(CategoryID),
    FOREIGN KEY (SupplierID) REFERENCES Supplier(SupplierID)
);

/* =====
===
3. ÍNDICES
=====
== */

-- Crear índices
CREATE INDEX IX_Category ON Product(CategoryID);
CREATE INDEX IX_Supplier ON Product(SupplierID);

```

GO

```
/* =====  
==  
4. DATOS DE EJEMPLO  
=====
```

```
== */
```

```
USE EcommerceDB;
```

GO

```
-- Insertar datos en Supplier
```

```
INSERT INTO Supplier (SupplierName, Country, Email, Phone)  
VALUES  
    ('Contoso Supplies', 'USA', 'contact@contoso.com', '555-  
0100'),  
    ('Fabrikam Inc', 'Canada', 'sales@fabrikam.com', '555-02  
00');
```

```
-- Insertar datos en Category
```

```
INSERT INTO Category (CategoryName, Description)  
VALUES  
    ('Electronics', 'Electronic devices and accessories'),  
    ('Clothing', 'Apparel and fashion items');
```

```
-- Insertar datos en Product
```

```
INSERT INTO Product (ProductName, CategoryID, SupplierID, Ba  
sePrice, StockQuantity)  
VALUES  
    ('Wireless Mouse', 1, 1, 29.99, 100),  
    ('Cotton T-Shirt', 2, 2, 19.99, 250);
```

GO

```
/* =====  
==
```

5. TABLAS TEMPORALES

```
=====
== */

-- Crear tabla temporal con los precios de los productos
CREATE TABLE ProductPrice (
    PriceID INT PRIMARY KEY IDENTITY(1,1),
    ProductID INT NOT NULL,
    CurrentPrice DECIMAL(10,2) NOT NULL,
    EffectiveDate DATE,
    SysStartTime DATETIME2 GENERATED ALWAYS AS ROW START HIDDEN,
    SysEndTime DATETIME2 GENERATED ALWAYS AS ROW END HIDDEN,
    PERIOD FOR SYSTEM_TIME (SysStartTime, SysEndTime),
    FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
) WITH (SYSTEM_VERSIONING = ON);
GO

-- Insertar datos iniciales en ProductPrice
INSERT INTO ProductPrice (ProductID, CurrentPrice, EffectiveDate)
VALUES (1, 99.99, '2025-01-01'), (2, 149.99, '2025-01-01');

-- Actualizar precios y crear entradas para el historial
UPDATE ProductPrice SET CurrentPrice = 109.99 WHERE ProductID = 1;
GO
```

Consulta sobre la tabla temporal ProductPrice, para ver por cada producto el histórico de precios con su respectiva fecha.

```
SELECT ProductID, CurrentPrice, SysStartTime, SysEndTime
FROM ProductPrice
```

```
FOR SYSTEM_TIME ALL
WHERE ProductID = 1;
```

Resultado:

100 % ✓ No se encontraron problemas.

Resultados Mensajes

	ProductID	CurrentPrice	SysStartTime	SysEndTime
1	1	109.99	2026-09-18 12:05:38.5194837	9999-12-31 23:59:59.9999999
2	1	99.99	2026-09-18 12:05:38.5184840	2026-09-18 12:05:38.5194837

Las columnas JSON permiten almacenar datos flexibles y variables que cambian según el tipo de producto, y la columna calculada que guarda el valor del Color como MetadataColor

```
-- Agregar columna de metadata a Product (JSON requiere SQL Server 2025)
```

```
ALTER TABLE Product ADD Metadata JSON;
GO
```

```
-- Agregar columna calculada indexada
```

```
ALTER TABLE Product ADD MetadataColor AS JSON_VALUE(Metadata, '$.color');
GO
```

```
-- Crear índice en la columna calculada
```

```
CREATE NONCLUSTERED INDEX IX_Product_Metadata_Color
ON Product (MetadataColor);
GO
```

```
-- Actualizar los productos con metadata
```

```
UPDATE Product SET Metadata = N'{"color":"blue","size":"large","material":"cotton"}'
WHERE ProductID = 1;
```

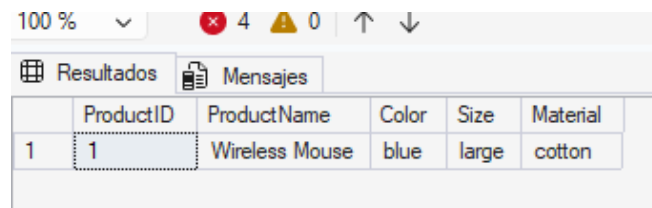
```
UPDATE Product SET Metadata = N'{"color":"red","size":"small","material":"cotton"}'
```

```
l", "material": "silk"}'
WHERE ProductID = 2;
GO
```

Consulta sobre la metadata sobre productos de color azul

```
SELECT
    ProductID,
    ProductName,
    JSON_VALUE(Metadata, '$.color') AS Color,
    JSON_VALUE(Metadata, '$.size') AS Size,
    JSON_VALUE(Metadata, '$.material') AS Material
FROM Product
WHERE JSON_VALUE(Metadata, '$.color') = 'blue';
```

Resultado:



	ProductID	ProductName	Color	Size	Material
1	1	Wireless Mouse	blue	large	cotton

Partición para la tabla Order para que los pedidos se distribuyan en distintas particiones según su fecha. También se crea un índice alineado.

```
-- Crear una función de partición basada en las fechas de los
pedidos
-- Usar RANGE RIGHT para mantener juntos los valores correspo
ndientes al mismo día
CREATE PARTITION FUNCTION PF_OrderDate (DATE)
AS RANGE RIGHT FOR VALUES
('2025-01-01', '2025-04-01', '2025-07-01', '2025-10-0
1');
```

```

-- Crear el esquema de partición utilizando el grupo de archivos PRIMARY
CREATE PARTITION SCHEME PS_OrderDate
    AS PARTITION PF_OrderDate ALL TO ([PRIMARY]);

-- Crear la tabla Order particionada
-- Incluir OrderDate en la clave primaria para alinear el índice con el particionado
CREATE TABLE [Order] (
    OrderID BIGINT IDENTITY(1,1),
    OrderDate DATE NOT NULL,
    CustomerName NVARCHAR(100) NOT NULL,
    TotalAmount DECIMAL(12,2) NOT NULL,
    OrderStatus NVARCHAR(20) DEFAULT 'Pending',
    CONSTRAINT PK_Order PRIMARY KEY (OrderID, OrderDate),
    CHECK (TotalAmount > 0),
    CHECK (OrderStatus IN ('Pending', 'Processing', 'Shipped', 'Delivered', 'Cancelled'))
) ON PS_OrderDate(OrderDate);

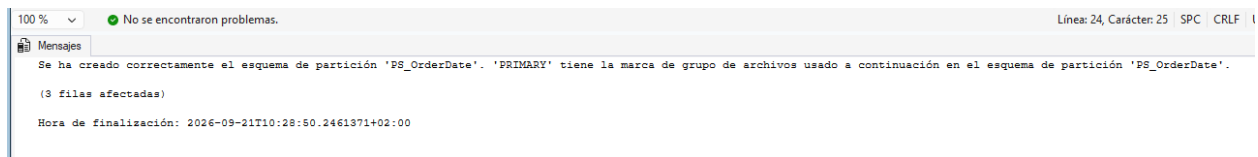
-- Crear un índice particionado
CREATE NONCLUSTERED INDEX IX_Order_Customer
    ON [Order](CustomerName)
    ON PS_OrderDate(OrderDate);

GO

-- Insertar pedidos de ejemplo
INSERT INTO [Order] (OrderDate, CustomerName, TotalAmount, OrderStatus) VALUES
    ('2025-01-15', 'John Smith', 299.97, 'Delivered'),
    ('2025-02-20', 'Jane Doe', 149.99, 'Shipped'),
    ('2025-06-10', 'Bob Johnson', 449.95, 'Processing');

GO

```



Se crea una secuencia para generar los identificadores de OrderDetail y se insertan líneas de pedido relacionadas con pedidos y productos.

```
/* =====
==
8. SECUENCIA Y DETALLE DE PEDIDOS
=====
*/
-- Crear una SEQUENCE para generar los identificadores de las
líneas de pedido
CREATE SEQUENCE OrderLineSequence
    START WITH 1
    INCREMENT BY 1;

-- Crear la tabla OrderDetail
CREATE TABLE OrderDetail (
    OrderLineID INT PRIMARY KEY,
    OrderID BIGINT NOT NULL,
    OrderDate DATE NOT NULL,
    ProductID INT NOT NULL,
    Quantity INT NOT NULL,
    UnitPrice DECIMAL(10,2) NOT NULL,
    LineTotal AS (Quantity * UnitPrice),
    CHECK (Quantity > 0),
    CHECK (UnitPrice > 0),
    FOREIGN KEY (OrderID, OrderDate) REFERENCES [Order](OrderID, OrderDate),
    FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
);
GO
```

```
-- Insertar detalles de pedidos utilizando la SEQUENCE
INSERT INTO OrderDetail (OrderLineID, OrderID, OrderDate, ProductID, Quantity, UnitPrice)
VALUES
    (NEXT VALUE FOR OrderLineSequence, 1, '2025-01-15', 1, 2, 99.99),
    (NEXT VALUE FOR OrderLineSequence, 1, '2025-01-15', 2, 1, 149.99),
    (NEXT VALUE FOR OrderLineSequence, 2, '2025-02-20', 1, 3, 99.99);
GO

SELECT * FROM OrderDetail;
```

Resultado:

	OrderLineID	OrderID	OrderDate	ProductID	Quantity	UnitPrice	LineTotal
1	1	1	2025-01-15	1	2	99.99	199.98
2	2	1	2025-01-15	2	1	149.99	149.99
3	3	2	2025-02-20	1	3	99.99	299.97
4	4	1	2025-01-15	1	2	99.99	199.98
5	5	1	2025-01-15	2	1	149.99	149.99
6	6	2	2025-02-20	1	3	99.99	299.97

Consultas para verificar que las restricciones, el JSON, el particionado y el historial temporal funcionan correctamente.

Comprobar que las restricciones funcionan en la tabla Product, se espera que falle porque el precio es negativo y se especificó **CHECK (BasePrice > 0)**

```
-- Esta inserción debe fallar porque el precio es negativo
INSERT INTO Product (ProductName, CategoryID, SupplierID, BasePrice, StockQuantity)
VALUES ('Invalid', 1, 1, -50, 10);
```

Resultado:

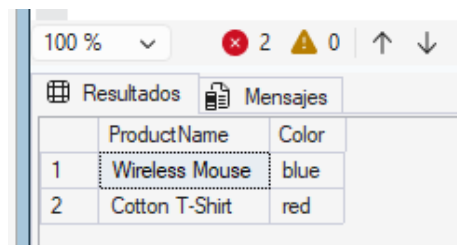
Mens. 547, Nivel 16, Estado 0, Línea 3
Instrucción INSERT en conflicto con la restricción CHECK 'CK_Product_BasePri_534D60F1'. El conflicto ha aparecido en la base de datos 'EcommerceDB', tabla 'dbo.Product', column 'BasePrice'.
Se terminó la instrucción.

Hora de finalización: 2026-09-21T10:39:12.7866864+02:00

Comprobar que las consultas sobre datos JSON funcionan correctamente

```
SELECT ProductName, JSON_VALUE(Metadata, '$.color') AS Color
FROM Product
WHERE Metadata IS NOT NULL;
```

Resultado:



The screenshot shows a SQL Server query results window. The 'Results' tab is active, displaying a table with two columns: 'ProductName' and 'Color'. There are two rows of data: 'Wireless Mouse' with 'blue' and 'Cotton T-Shirt' with 'red'. The window also shows a status bar with '100 %' zoom, '2' errors, '0' warnings, and navigation arrows.

	ProductName	Color
1	Wireless Mouse	blue
2	Cotton T-Shirt	red

```
-- Comprobar la distribución de los registros entre las parti-
ciones
SELECT $PARTITION.PF_OrderDate(OrderDate) AS Partition, COUNT(*) AS RecordCount
FROM [Order]
GROUP BY $PARTITION.PF_OrderDate(OrderDate);
```

Resultado:

100 % ✖ 1 ⚠ 0 ↑

Resultados Mensajes

	Partition	RecordCount
1	2	4
2	3	2



Los mensajes de "error" en rojo son IntelliSense desactualizado, no errores de ejecución.

Comprobar el historial de la tabla temporal SELECT ProductID, CurrentPrice, SysStartTime, SysEndTime

```
FROM ProductPrice FOR SYSTEM_TIME ALL
ORDER BY ProductID, SysStartTime;
```

Resultado:

Resultados Mensajes

	ProductID	CurrentPrice	SysStartTime	SysEndTime
1	1	99.99	2026-09-18 12:05:38.5184840	2026-09-18 12:05:38.5194837
2	1	109.99	2026-09-18 12:05:38.5194837	9999-12-31 23:59:59.9999999
3	2	149.99	2026-09-18 12:05:38.5184840	9999-12-31 23:59:59.9999999